CGV Seminar:
The **Implementation** component

CS4705 Research Seminar Computer Graphics
DSAIT4215 Research in Visual Computing and Interactive Visual Analysis

2025/2026

Summary

This document contains instructions for realization of the Implementation component of the CGV Seminar course (CS4705/DSAIT4215). Please read the complete document carefully before you start.

Objectives

The Implementation component supplements the paper presentation. It should provide additional exposition, explanation or exploration of the paper topics. Therefore, the goal of the Implementation is explicitly **NOT** to simply reimplement the paper. Instead, you create an artifact that would be useful as an **Educational tool**. It should support exposition of the paper such that it could be used alongside paper presentation for better explanation. It does not need to cover all aspects of the paper and may instead focus on one or few elements. Some (non-exhaustive) guiding questions: What is difficult to understand and could be illustrated better? How can you make the inner workings of a step accessible? Is it possible to illustrate the importance of parametric or algorithmic choices?

Connection to the course

The Implementation component is used to assess completion of the following learning objectives:

- LO3: Implement a tool that helps to further explain the discussed technique.
- LO4: Critically assess results of the implementation in form of a short report and a presentation.

Preliminaries

The implementation must be connected to the paper selected for the Presentation component. Note that if you selected a custom paper (Option 2), you need **an approval from a course instructor** (a "Supervisor"). During the paper selection, you should also consider your implementation options. In all cases, it is **your responsibility** to design and execute an implementation plan that fulfils the course objectives.

Deliverables

The solution must be uploaded to Brightspace -> Assignments -> Implementation by the deadline displayed on Brightspace. Submissions outside Brightspace will not be graded.

A late submission will be **penalized**:

$$penalized\_grade = original\_grade - ceil\left(\frac{minutes\ late}{10}\right)$$

Submit everything as a single ZIP file containing:

- **Code**: A zip file with code, data and readme.
- **Report**: A PDF.
- **Video-presentation**: An mp4.

Make sure to remove any unnecessary binaries and large data files. In general, your submission should not be larger than 100-200 MiB but we leave it to your own judgment. You can provide instructions on how to download any larger data in your readme and only include minimal data sample for testing in the BS submission.

Technical requirements

Use of **external resources,** such as code repositories or libraries, is allowed if it is clearly reported with a proper citation and following all other rules in this document. Note that **we only grade your own work** and not work done by others so extensive reliance on external code may diminish your own contribution to the solution.

See the course "Organization slides" for **Policy on use of Generative AI** on Brightspace.

1) Code

You are free to propose any technical solution (e.g. coding language) as a part of your implementation plan.

If you use any external code, you must separate your code and external code to separate folders. All external code must be isolated in a single root sub-folder "**`external_code`**". We will not grade the code in this folder. You are allowed to modify the code in "**`external_code`**", but it will still not be graded.

The root directory of your submission must contain a **README.md** file with:

- Setup instructions – how to compile, what to download, …
- Run instructions – an exact command to type to run your program
- External resources – an itemized listing of **all external resources** with proper references and explanation of what they were used for. This includes imported code as well as included binary libraries or any other sources (data, pre-trained weights, …)

2) Report

An extended abstract with **maximum of 1 page of text** and maximum of 1 page of Figures submitted as a PDF.

3) Video

A video-presentation of the Implementation submitted as an mp4 (we recommend x264/H264 codec). If your solution is an application, the video should **show the application running and demonstrate/explain its (correct) functionality**. The intended length is **3 minutes** and, we will only watch the first 3 minutes and ignore any additional footage.

Assessment

The Implementation component constitutes 40% of the final grade and as such it represents a substantial share of the course work. For a 5 EC course, the expected workload attributed to the Implementation component is 56 hours (0.4 x 5 x 28 hours). Keep this in mind when designing your solution.

The solution will be graded based on the **educational value**, **code**, **report** and **video**.

Minimal functional requirements

Fulfilling the technical requirements and the minimal functional requirements is **a necessary but not a sufficient** condition for obtaining a grade >= 5.0.

- The solution must have a substantial **educational value** and be useful for explanation of the assigned paper or its part.
    - **Bad example**: An ablation study resulting in a table of numbers does not help us understand the paper.
    - **Counter example**: A tool visualizing steps of an algorithm that does help us understand the paper.
- The solution must be substantially based on **custom code** (external code is not considered during grading). Consider this when selecting the paper.
    - **Bad example:** A method for neural image classification. Cloned the author's git and made edits to divert network activations into png files as visualizations.
    - **Counter example:** Prior year examples shared in the lecture.
- The **report** must be comprehensible. It must explain 1) What was done, 2) How it connects to the presented paper and 3) How the product helps to explain the paper or its part(s) (i.e., how would you use it as an educational tool).
    - **Bad example:** A report shows a list of numbers, a couple screenshots and mentions that the algorithm was optimized for better runtime without explaining how and why.
    - **Counter example:** The report explains that a visualization was made to explain how geometric transformations are done as a part of the data processing and that students can experiment with them interactively using a GUI which helps them to get a good understanding of the algorithm behavior. This is supported by use-case figures with clear captions.
- The **video** must show the program running and showcase the key functionality.
    - **Bad example**: Most of the footage is spent looking at a loading screen, then a complicated GUI appears which is not explained. The user clicks buttons, but it is not clear what is happening and why.
    - **Counter example**: A clearly narrated video showing the program, explaining what is being done and why.